

JavaScript

Clases y Programación Orientada a Objetos

Resumen explicado y conciso

Temas: clases, objetos, constructor, this, métodos, privacidad, static, getters/setters, herencia y polimorfismo.

Idea central

JavaScript está basado en prototipos. La sintaxis `class` no reemplaza ese modelo: lo presenta de una forma más familiar y clara para trabajar con objetos y herencia.

1. ¿Qué significa “clase” en JavaScript?

El material parte de una distinción importante: JavaScript es un lenguaje basado en prototipos, pero permite escribir clases. Esas clases son una capa sintáctica más cómoda sobre la herencia prototípica, no un modelo de objetos completamente distinto.

Concepto	Qué significa
Prototipos	Los objetos pueden derivar comportamiento y propiedades de otros objetos sin que sea obligatorio definir primero una clase explícita.
class	Sintaxis más clara para definir el “molde” con el que construiremos objetos. Internamente sigue apoyándose en prototipos.
Objeto / instancia	Un objeto concreto creado a partir de una clase.

Constructor y creación de objetos

El método especial constructor se ejecuta al crear una instancia. Se usa para inicializar sus propiedades. La instancia se crea con new.

```
class Persona {
  constructor(nombre, apellido) {
    this.nombre = nombre;
    this.apellido = apellido;
  }
}

let javy = new Persona("Javier", "Baez");
```

Clave: `this.nombre` es la propiedad del objeto que se está creando; `nombre` es el parámetro recibido por el constructor.

2. Propiedades y métodos

Las propiedades almacenan el estado de cada objeto. Los métodos son funciones asociadas al objeto y operan sobre ese estado.

```
class Persona {  
    constructor(nombre, apellido) {  
        this.nombre = nombre;  
        this.apellido = apellido;  
    }  
  
    nombreCompleto() {  
        return this.apellido + ", " + this.nombre;  
    }  
}
```

Modelar datos que cambian

El PDF usa la edad como ejemplo. Guardar directamente edad es incómodo porque el valor cambia con el tiempo. La alternativa presentada es guardar fechaNacimiento y calcular la edad cuando se necesita.

La idea de diseño es simple: si un valor puede derivarse de otro dato más estable, conviene guardar el dato estable y calcular el derivado.

Ejemplo de propiedad calculada con un getter:

```
class Persona {  
    // ... constructor con fechaNacimiento  
  
    get edad() {  
        return parseInt(  
            (new Date() - this.fechaNacimiento) / 31536000000  
        );  
    }  
}  
  
console.log(javy.edad);
```

3. Público, privado y estático

Campos públicos y privados

El material indica que los campos son públicos por defecto. Para declarar un campo privado se antepone #. Ese campo no puede accederse directamente desde fuera de la clase.

```
class Persona {  
    #dni;  
  
    constructor(nombre, apellido, dni) {  
        this.nombre = nombre;  
        this.apellido = apellido;  
        this.#dni = dni;  
    }  
}
```

Propiedades y métodos estáticos

static define miembros que pertenecen a la clase, no a cada instancia. Por eso se acceden usando el nombre de la clase.

```
class Persona {  
    static total = 0;  
  
    constructor(nombre, apellido) {  
        this.nombre = nombre;  
        this.apellido = apellido;  
        Persona.total = Persona.total + 1;  
    }  
  
    static totalPersonas() {  
        return Persona.total;  
    }  
}  
  
Persona.totalPersonas();
```

Un método estático no necesita estar vinculado a una propiedad estática: simplemente es un método que se invoca desde la clase.

4. Getters y setters

Son propiedades de acceso: se usan como si fueran propiedades, pero detrás ejecutan una función.

Elemento	Cuándo se ejecuta
get	Cuando se lee o consulta la propiedad.
set	Cuando se intenta asignar un valor a la propiedad.

```
class ClaseQueUtilizaGetSet {
  #msg = "hello world";

  get msg() {
    return this.#msg;
  }

  set msg(x) {
    this.#msg = `hello ${x}`;
  }
}

const instance = new ClaseQueUtilizaGetSet();
instance.msg;           // llama al getter
instance.msg = "cake"; // llama al setter
```

Nota: el slide original usa nombres de clase distintos al declarar e instanciar; aquí se unificó el nombre únicamente para que el ejemplo sea legible.

Qué aportan

- Permiten controlar cómo se obtiene un valor.
- Permiten controlar qué ocurre cuando se asigna un valor.
- Sirven para exponer un dato calculado, como la edad obtenida a partir de la fecha de nacimiento.

Desde fuera, el uso sigue pareciendo el acceso a una propiedad: `obj.edad` en vez de una llamada explícita a un método.

5. Herencia: crear una subclase

Una subclase parte de una clase existente. Hereda sus propiedades y comportamiento, y puede agregar nuevos datos o métodos.

```
class Empleado extends Persona {
    constructor(nombre, apellido, fechaNacimiento, sueldo) {
        super(nombre, apellido, fechaNacimiento);
        this.sueldo = sueldo;
    }

    detalleSueldo() {
        return this.apellido + " cobra: " + this.sueldo;
    }
}
```

Sintaxis	Función
<code>extends</code>	Declara que una clase hereda de otra.
<code>super(...)</code>	Invoca el constructor de la superclase para inicializar la parte heredada del objeto.

¿Y el polimorfismo?

El PDF lo incluye dentro de los temas de la clase y propone ejercicios de “Empleados” y “Movimientos de Ajedrez”, pero no desarrolla una definición teórica ni un ejemplo completo. Por eso este resumen no agrega una explicación externa al material.

Repaso express

Si recordás estas relaciones, tenés el núcleo del PDF:

Sintaxis	Idea
<code>class</code>	Define la estructura y comportamiento con una sintaxis cómoda.
<code>constructor(...)</code>	Inicializa una nueva instancia.
<code>new Clase(...)</code>	Crea un objeto de esa clase.
<code>this.x</code>	Accede a la propiedad x del objeto actual.
<code>metodo() { ... }</code>	Define comportamiento de las instancias.
<code>#campo</code>	Declara un campo privado.
<code>static</code>	Hace que el miembro pertenezca a la clase, no a cada objeto.
<code>get x()</code>	Calcula/entrega x cuando se lee como propiedad.
<code>set x(v)</code>	Ejecuta lógica cuando se asigna x.
<code>extends</code>	Crea una subclase a partir de otra clase.
<code>super(...)</code>	Inicializa la parte heredada usando el constructor padre.

Mapa mental

Clase → define cómo crear objetos → constructor inicializa → this apunta al objeto → métodos trabajan con sus propiedades → # oculta campos → static pertenece a la clase → get/set controlan acceso → extends/super permiten herencia.

Resumen elaborado únicamente a partir del material proporcionado.